

Building a Reproducible Numerai Quant-ML Research Pipeline

Nandhu Raj

June 2026

Abstract

This document summarizes the design and iteration history of a reproducible Numerai research pipeline for cross-sectional tabular prediction. The project began as a simple LightGBM baseline and evolved into a multi-model experiment system with era-aware walk-forward validation, feature exposure diagnostics, post-prediction neutralization, cached ensemble reblending, live prediction generation, and dry-run submission support. The goal was not to claim profitability, but to build a practical research workflow that reflects realistic quant-ML and ML engineering practice. This writeup focuses on what was implemented, the failures encountered, the fixes applied, the alternative paths explored, and what the current results support.

1 Project Goal

The project objective was to build a Numerai pipeline that could support repeatable experimentation instead of one-off model runs. The intended workflow was:

1. download Numerai tournament data without committing raw files;
2. train baseline and alternative tabular models;
3. validate them with era-aware walk-forward backtesting;
4. generate live predictions in Numerai submission format;
5. support dry-run-first submission through environment variables rather than hardcoded credentials;
6. leave behind reproducible artifacts, plots, and reports after every run.

The emphasis was therefore on making the research loop systematic, inspectable, and reusable rather than relying on a single model run.

2 Problem Setting

Numerai provides obfuscated market data with anonymous features, an **era** column, row identifiers, and a target. The practical modeling task is to predict the target cross-sectionally while respecting the time-like structure implied by eras. Since the features are anonymized, domain-specific feature engineering is limited. That naturally shifts the emphasis toward:

- validation design,

- model comparison,
- ensemble construction,
- exposure control,
- and workflow discipline.

The project used the Numerai dataset version **v5.2** and local experiments were primarily run under Python 3.12 with **uv** for environment management.

3 System Design

3.1 Repository Structure

The repository was organized as a small Python package rather than a notebook-first project. Core logic lives under **src/numerai_quant/**, command-line entry points live under **scripts/**, experiment settings live under **configs/**, and run outputs are written into ignored **artifacts/**, **models/**, and **data/** directories.

3.2 Implemented Components

The final system includes:

- Numerai data download through **numerapi**;
- automatic feature detection from columns prefixed with **feature_**;
- LightGBM, XGBoost, CatBoost, and MLX-based MLP model support;
- walk-forward fold generation using ordered eras, embargo periods, and configurable fold counts;
- validation metrics including era-wise Spearman correlation, mean correlation, standard deviation, Sharpe-like score, drawdown-style diagnostic, and feature exposure;
- post-prediction neutralization against highly exposed features;
- saved checkpoints and partial artifacts for interrupted runs;
- cached reblending from stored fold predictions without retraining base models;
- final ensemble bundle export for live inference;
- live prediction formatting and dry-run submission flow.

4 Validation Protocol

Validation was the central methodological choice in the project. Instead of using random train/validation splits, the pipeline treats **era** as a time-like grouping and evaluates models in a walk-forward fashion:

1. train on earlier eras;

2. skip an embargo gap to reduce leakage;
3. validate on later eras;
4. repeat across multiple folds.

This does not eliminate all backtest risk, but it is far more defensible than independent and identically distributed splits for this setting.

For fast local iteration, many experiments used a medium validation budget with 3 folds and 4 validation eras per fold, yielding 12 validation eras in total. Later, stronger benchmark configurations were added with 15 folds and the same 4-era validation window, yielding 60 validation eras.

5 Experiment Chronology

5.1 Stage 1: Baseline Trees

The project started with LightGBM as the baseline system. This established the initial pipeline shape: loading parquet data, auto-detecting feature columns, training on the Numerai target, and producing rank-normalized predictions in the valid range.

Two LightGBM variants were retained in the model zoo:

- `lgbm_main`, which became the stable tree baseline;
- `lgbm_alt`, which provided a weaker but still useful alternative signal for ensembles.

5.2 Stage 2: XGBoost Branch

The next experiment branch added XGBoost as a third model family. On the medium-scale validation setup, the XGBoost run produced:

Model	Mean CORR	Sharpe-like
ensemble	0.075227	1.084128
xgb_optional	0.071617	1.185753
lgbm_main	0.070495	1.045938
lgbm_alt	0.052972	0.977822

This was a meaningful improvement over the original LightGBM-only setup and established that a model zoo plus ensembling was already better than a single-model baseline.

5.3 Stage 3: CatBoost Branch

CatBoost was then added as another alternative model family. Early CatBoost settings underperformed badly. A representative early medium run yielded:

Model	Mean CORR	Sharpe-like
lgbm_main	0.070495	1.045938
ensemble	0.068955	1.044147
ensemble_neutralized	0.067549	1.015856
lgbm_alt	0.052972	0.977822
catboost_optional	0.038138	1.067775

That poor first result was useful because it forced parameter iteration instead of letting the project tell an overly tidy story. After tuning, the CatBoost branch improved substantially:

Model	Mean CORR	Sharpe-like
catboost_optional	0.079392	0.928660
ensemble	0.078841	1.015293
ensemble_neutralized	0.077548	0.994846
lgbm_main	0.070495	1.045938
lgbm_alt	0.052972	0.977822

At this point CatBoost became the strongest standalone non-LightGBM model in the repository.

5.4 Stage 4: Cached Reblending

One of the strongest engineering improvements in the project was not a new model at all. It was cached reblending.

The walk-forward pipeline had already been modified to save fold-level prediction checkpoints. Building on that, a new workflow was added to recompute ensemble results from stored fold predictions without retraining any base models. This made it much cheaper to test new weights and blend strategies.

Optimizing the CatBoost-heavy blend on cached predictions produced:

Model	Mean CORR	Sharpe-like
ensemble	0.081843	0.972208
ensemble_neutralized	0.080924	0.957781
catboost_optional	0.079392	0.928660
lgbm_main	0.070495	1.045938
lgbm_alt	0.052972	0.977822

The best optimized CatBoost reblend chose weights:

$$\text{lgbm_main} = 0.30, \quad \text{lgbm_alt} = 0.00, \quad \text{catboost_optional} = 0.70$$

This was an important moment in the project because it showed that workflow improvements could create measurable modeling gains.

5.5 Stage 5: Feature Filtering Experiments

The project also explored simple feature-selection and redundancy-control paths, including:

- correlation pruning;
- importance-based top- N filtering.

These were implemented cleanly and evaluated, but in practice they hurt performance on the medium validation setup. That result is worth documenting because it prevented unnecessary complexity from becoming permanent just because it sounded sophisticated.

5.6 Stage 6: MLX Neural Branch

Because the hardware environment included Apple Silicon, the project added an MLX-based MLP path as a neural tabular experiment lane.

The first MLX attempt was weak. Later versions improved sharply, especially after the training loop was aligned more closely with Numerai validation:

- early stopping on the actual fold validation set instead of only a random internal split;
- support for validation based on era-wise correlation rather than only MSE;
- input noise and larger hidden layers in later configurations.

An improved MLX-inclusive run produced:

Model	Mean CORR	Sharpe-like
ensemble	0.082707	1.262571
ensemble_neutralized	0.082453	1.234679
lgbm_main	0.070495	1.045938
mlx_mlp_optional	0.058747	3.461954
lgbm_alt	0.052972	0.977822

This was the best medium-scale ensemble score observed in the project. However, the MLX model itself was not the strongest standalone learner. Its value appeared to come from diversification rather than raw standalone accuracy.

When the same MLX branch was optimized with cached reblending, the result fell to:

Model	Mean CORR	Sharpe-like
ensemble	0.080167	1.170258
ensemble_neutralized	0.079942	1.143904
lgbm_main	0.070495	1.045938
mlx_mlp_optional	0.058747	3.461954
lgbm_alt	0.052972	0.977822

The optimized weights for that branch were:

$$\text{lgbm_main} = 0.55, \quad \text{lgbm_alt} = 0.05, \quad \text{mlx_mlp_optional} = 0.40$$

This made the MLX branch more interesting, not less. It suggested that the neural model may be adding useful but less stable diversity, rather than providing a robust replacement for the tree-based baseline.

6 Engineering Issues Encountered and Fixes Applied

The repository improved substantially through debugging and workflow repair, not only through model tuning.

6.1 Data Download Friction

Initially, the downloader pulled train, validation, and live files sequentially every time. That created unnecessary waiting and made failures harder to reason about. The data workflow was revised to:

- support selective flags such as `-train-only`, `-validation-only`, and `-live-only`;
- separate dependency-aware steps so backtests can start as soon as training data is ready;
- log clear start and finish messages around each file;
- reuse a single NumerAPI client.

6.2 Interrupted Runs and Partial Progress

Long runs on a laptop and in Colab were vulnerable to crashes, disconnects, and sleep-related interruptions. To reduce wasted time, the pipeline was extended to write:

- partial fold metrics;
- partial OOF predictions;
- fold checkpoints;
- run status metadata.

This made it much easier to inspect what had completed and enabled the later reblend workflow.

6.3 Identifier and Index Handling

At one point the backtest failed with a missing `id` column error. The root cause was that some frames treated the identifier as an index rather than a normal column. The fix was to centralize identifier handling in a helper that promotes the index to a named identifier column when needed.

6.4 Backend and Compatibility Issues

The project also hit a plotting backend failure in Colab because `matplotlib_inline` was being inherited as a backend value unsupported by the package configuration in that environment. The initialization path was updated to behave more safely in notebook and Colab-style contexts.

Another small but disruptive issue came from using a `reset_index(names=...)` pattern that was not supported in the runtime mix actually used during execution. That was simplified to a more compatible approach.

6.5 MLX Validation Mismatch

The early MLX implementation used a random internal validation split with MSE-based early stopping. That was a poor match for the actual evaluation target. The fix was to let the MLX trainer use the fold validation slice directly and to support era-wise correlation as the validation signal.

7 What the Results Support Right Now

The current repository supports several honest claims:

1. The project is no longer a toy notebook. It is a reproducible, config-driven ML research pipeline.
2. Era-aware backtesting was implemented correctly enough to make model comparisons more credible than random-split baselines.
3. Ensembling across model families was useful, and cached reblending improved iteration speed substantially.
4. CatBoost became the strongest stable tree-based alternative beyond LightGBM.
5. MLX-based neural models became competitive enough to improve a medium-scale ensemble result, even if they are not yet the most stable branch.

At the same time, the repository does *not* support stronger claims such as:

- proven profitability;
- live robustness;
- production-grade monitoring;
- or a definitive statement that the neural branch is superior to the tree branch.

The medium sample run still uses 12 validation eras, which is useful for iteration and documentation but not enough for a strong claim on its own. Stronger benchmark configurations with 60 validation eras have already been added to address this.

8 Alternative Paths Explored

Several alternative directions were evaluated during the project:

- **XGBoost branch:** useful and competitive, but ultimately overtaken by the best CatBoost configuration.
- **CatBoost branch:** initially weak, then improved enough to become the strongest stable branch.
- **Feature filtering:** implemented cleanly but not retained as a winning direction because it hurt scores.
- **Neural MLX branch:** more volatile than the tree-based alternatives, but valuable because it introduced a genuinely different signal family.
- **Manual vs. optimized blending:** both were informative. In the CatBoost branch, optimized reblending improved the score. In the MLX branch, the manual blend outperformed the optimizer, indicating lower stability.

9 Operational Constraints

The experiments were shaped by very real hardware and workflow constraints:

- large parquet downloads are slow and storage-heavy;
- a MacBook Pro can run these experiments, but long multi-fold training creates thermal and unified-memory pressure;
- Colab can help, but session disconnects and ephemeral files make artifact persistence fragile unless runs are structured carefully.

Those constraints pushed the project toward a more disciplined approach to checkpointing, selective downloads, thread capping, and cached re-use of fold predictions.

10 Current Best View of the System

At the time of writing, the project has two strong research branches:

- a **CatBoost-heavy branch**, which is the strongest stable benchmark path;
- an **MLX-inclusive branch**, which may produce higher medium-run ensemble scores but has not yet shown the same robustness under blend optimization.

The practical interpretation is:

- CatBoost is the safer branch for a stronger benchmark run;
- MLX is the more experimental branch, worth keeping because it can add useful diversity.

11 Next Steps

The next steps are straightforward:

1. run the 60-validation-era benchmark configurations and replace medium-scale claims with stronger evidence;
2. re-optimize blends on those stronger runs;
3. decide whether the public benchmark should remain CatBoost-heavy or whether the MLX branch survives at larger scale;
4. eventually track live submission behavior rather than only backtest outputs.

12 Conclusion

This project started as a baseline Numerai modeling pipeline and improved through repeated iteration on both modeling and engineering. The most useful lesson was that experimentation infrastructure mattered as much as model choice. Better downloads, better checkpoints, better validation alignment, and cached rebinding changed the pace and quality of iteration. The resulting system is not proof of alpha, but it is a credible applied research workflow for quant-ML and ML engineering practice.

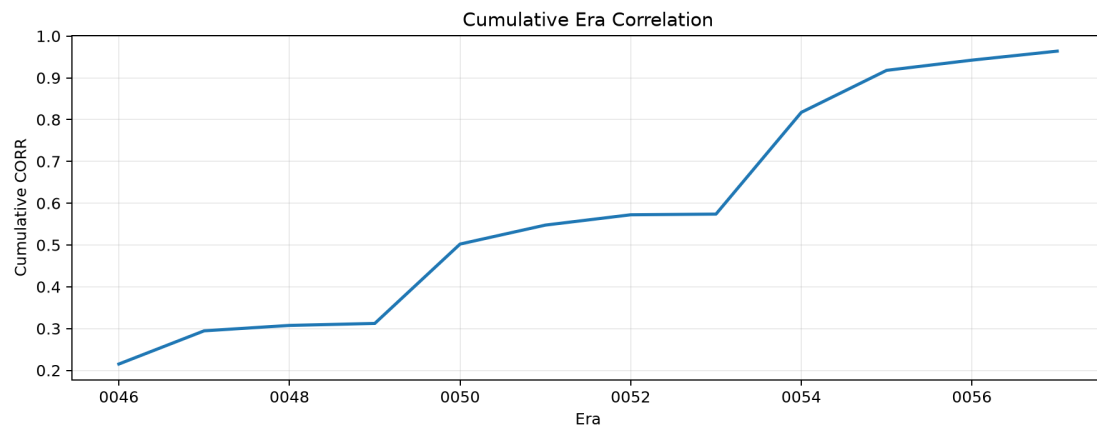


Figure 1: Cumulative era correlation from the curated medium-scale sample run.